ЛАБОРАТОРНАЯ РАБОТА

по дисциплине «Алгоритмы и структуры данных»

ТЕМА: «АЛГОРИТМЫ СЖАТИЯ БЕЗ ПОТЕРЬ»

Автор: Кутепов П.О.

СПбГЭТУ «ЛЭТИ», кафедра САПР

2025

Задание на лабораторную работу

- Исследовать зависимость энтропии сообщения от размера блоков, на которые разбивается текст, подаваемый на вход BWT+MTF, для тестовых файлов. Вывести полученные результаты на график, сравнить с исходной энтропией и сделать вывод об оптимальном размере блока.
- Исследовать зависимость коэффициента сжатия от размера буфера для алгоритма LZSS. Вывести полученные результаты на график и сделать вывод об оптимальном размере буфера.
- Исследовать зависимость коэффициента сжатия от размера словаря для алгоритма LZW. Вывести полученные результаты на график и сделать вывод об оптимальном размере словаря.
- Собрать на основе реализованных алгоритмов следующие компрессоры: HA, Run-length encoding (RLE), BWT + RLE, BWT + MTF + HA, BWT + MTF + RLE + HA, LZSS, LZSS + HA, LZW, LZW + HA.
- Выбрать размер блока для BWT на основе проведенных исследований.
- Для алгоритмов семейства LZ выбрать размер буфера/словаря на основе проведенных исследований.
- Удостовериться в корректной компрессии и декомпрессии данных для всех компрессоров.
- Исследовать эффективность компрессоров для всех тестовых данных. Свести размеры исходных файлов, файлов после компрессии и файлов после декомпрессии в байтах и соответствующие коэффициенты сжатия в таблицу.

Теоретическая часть

RLE

RLE кодирует повторяющиеся символы комбинацией символ + количество повторений. Пример: «aaabbbbccccc» -> «3a4b5c». Алгоритм можно усовершенствовать, резервируя старший бит в байте для маркировки единичных и повторяющихся символов («abcdeeeeefff» -> «^{0}4abcd^{1}5e^{1}3f»).

Для реализации базовой версии RLE, нужно просто последовательно читать символы. Если следующий символ совпадает с текущим, инкрементируем счётчик. Когда попадается символ, отличный от текущего, записываем текущий символ и его количество в результат.

Реализация со старшим битом не сильно отличается от базовой. Добавляется ещё один счётчик для неповторяющихся символов, который записывается в результат с единицей в начале байта (это можно сделать, используя побитовую операцию: «| 0x80»).

	T(n)	Пояснение
Best	$O(n)$	Один проход по данным
Average		
Worst		
	S(n)	Пояснение
Best	$2 = O(1)$	Один символ, «ааааааа»
Average	$\frac{2n}{k} = O\left(\frac{n}{k}\right),\ k$ – средняя длина серии	Пример: «ааааccbb», $k \approx 2{,}6$; $\frac{2n}{k} \approx 6$; Проверяем: «ааааccbb» -> «4a2c2b»
Worst	$2n = O(n)$	Чередующиеся символы, «abababab»

Таблица 1 – Кодирование RLE

	T(n)	Пояснение
Best	$O(n)$	Один проход по данным
Average		
Worst		
	S(n)	Пояснение
Best	$\frac{n}{2} = O(n)$	Чередующиеся символы, «abababab»
Average	$\frac{nk}{2} = O(nk),\ k$ – средняя длина серии	На один символ 2 байта (он сам + количество), после – повторение его k раз

Worst	$(2^{(8k)}-1)\frac{n}{2} = O(n2^k), k$ – Кол-во байт для записи повторов	Максимальное количество повторений на символ

Таблица 2 – Декодирование RLE

MTF

Цель алгоритма - преобразовать строку в последовательность маленьких повторяющихся чисел, чтобы потом эффективно сжать её с помощью других алгоритмов (Например, алгоритмом Хаффмана).

Для алгоритма необходимо иметь словарь проиндексированных символов. Для каждого символа заносится его индекс в строку с результатом, затем найденный символ перемещается в начало словаря (становится под индексом 0). При декодировании символы также перемещаются по словарю.

	T(n)	Пояснение
Best	$O(n)$	Один проход по данным + линейный поиск по словарю
Average	$o(nk)$	
Worst	$O(nk)$, k – размер алфавита	
	S(n)	Пояснение
Best	$O(n+k), k$ - размер алфавита	Хранение результата + алфавита
Average	$O(n+k)$	
Worst	$O(n+k)$	

Таблица 3 – Кодирование и декодирование MTF

Huffman algorithm

Алгоритм использует переменные длины кодов для символов: самым частым символам присваивается короткий код, редким – длинный. Вначале определяется частота символов, затем создаётся список нод (содержат сам

символ, его частоту, левого и правого потомка), из которых строится дерево. Список нод сортируется по частотам, выбираются 2 ноды, которые объединяются родителем, затем родитель добавляется в список. Процесс продолжается до тех пор, пока в списке не останется одна нода – корень дерева. Идя по этому дереву в левую ветвь, к коду добавляется 0, в правую – 1. В итоге получаем коды для всех символов, которые можно собрать в один словарь, чтобы быстро закодировать данные. В результирующую строку сохраняется не только последовательность кодов, но и дополнительная информация: длина словаря и словарь, состоящий из кодов, их длины и символов (также можно хранить просто символ и длину его кода). Для декодирования нужно восстанавливать дерево, после по битам прочитать строку: если бит 1 – идём в правую ветвь, 0 – в левую, у ноды больше нет потомков (в дереве Хаффмана у ноды не может быть единственный потомок) – выводим в результат.

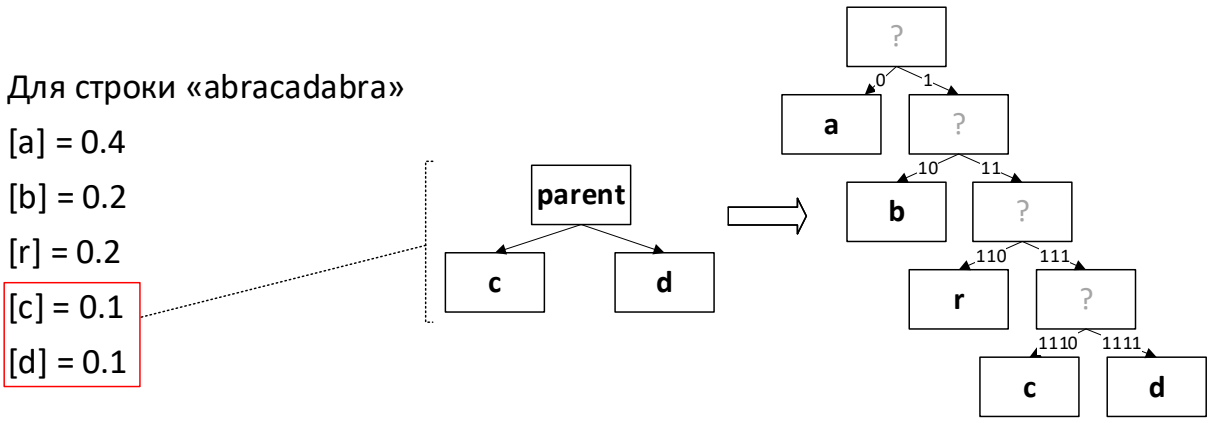


Рисунок 1 – Дерево Хаффмана

	T(n)	Пояснение
Best	$O(n) + O(k \log k) + O(k) + O(n) = O(n + k \log k), k -$ размер алфавита	Подсчёт частот символов, построение дерева, генерация кодов, кодирование
Average		
Worst		
	S(n)	Пояснение
Best		Хранение дерева, таблицы кодов и результата
Average		

Worst	$(2k-1)+O(kl)+O(n\frac{l}{8})=$ $O(n+k), k$ – размер алфавита, l – средняя длина кода	

Таблица 4 – Кодирование алгоритмом Хаффмана

	T(n)	Пояснение
Best	$O(k)+O(k\log k)+\frac{n}{l}O(\log k)$ $=O(n\log k)$	Чтение словаря, восстановление дерева, декодирование
Average		
Worst		
	S(n)	Пояснение
Best	$(2k-1)+O(kl)+O\left(n\frac{l}{8}\right)$ $=O(n+k)$	Хранение дерева, кодов и результата
Average		
Worst		

Таблица 5 – Декодирование алгоритма Хаффмана

BWT

Преобразование Барроуза – Уилера перестраивает строку так, что одинаковые символы встают рядом. Это позволяет эффективнее применять алгоритмы сжатия (например, RLE).

Создаётся матрица сдвигов. С изначальной строки первый символ переносится в конец – делаем так до тех пор, пока не получим все сдвиги. Затем сортируем сдвиги в лексикографическом порядке. В результат заносим последний столбец матрицы и номер исходной строки. Чтобы декодировать строку, восстанавливаем матрицу: в начало строк добавляем последний столбец и сортируем их. Так же восстановить строку можно отображением первого столбца в последний (рис. 2). Первый столбец получаем сортировкой последнего столбца. Начинаем с индекса целевой строчки – 2. В первом столбце под 2 – «B». По отображению 2 ведёт к 1 – «A», 1 – к 3 «A», 3 – к 0 «A».

Для строки «baca» (Нужная строка - вторая)

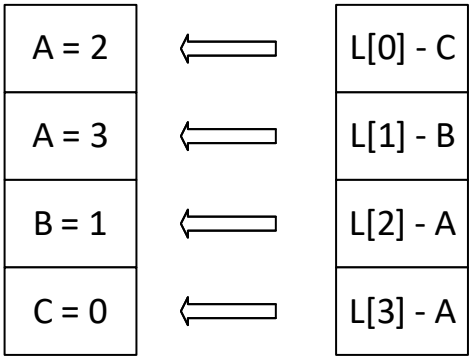


Рисунок 2 – LF отображение

	T(n)	Пояснение
Best	$O(n^2) + O(n^2) + O(n) + O(n) = O(n^2)$	Создание матрицы сдвигов, её сортировка, поиск исходной строки в матрице, получение последнего столбца матрицы
Average	$O(n^2) + O(n^2) + O\left(n\frac{n}{2}\right) + O(n) = O(n^2)$	
Worst	$O(n^2) + O(n^2) + O(n^2) + O(n) = O(n^2)$	
	S(n)	Пояснение
Best	$O(n^2)$	Хранение матрицы сдвигов и результата
Average		
Worst		

Таблица 6 – Кодирование BWT

	T(n)	Пояснение
Best	$O(n) + O(n\log n) + O(n) + O(n) = O(n\log n)$	Инициализация первого столбца, сортировка первого столбца, создание отображения, декодирование
Average		
Worst		
	S(n)	Пояснение
Best	$O(n)$	Хранение первого, последнего столбца матрицы и результата
Average		
Worst		

Таблица 7 – Декодирование BWT LF отображением

LZSS

LZSS – модернизированная версия LZ77. Оба позволяют кодировать повторения целых фраз в строке.

В процессе кодирования помимо входной строки мы передвигаем буфер, в котором ищем повторения. Берём символ, начинаем искать в буфере такой же. Если нашли – сравниваем последующие символы. Выбираем наибольшую длину совпадения и записываем в результат смещение до первого совпавшего символа и длину повторения. В LZ77 дополнительно записывался следующий символ, т.е. для строки «baraban» будет «00b00a00r21b21n». В LZ77 всё кодируется 3 байтами – триплетами. Если все символы различны, сжатие LZ77 наоборот увеличит строку в 3 раза. Но LZSS разделяет единичные символы, используя биты маркеров, которые можно записывать перед блоком, который содержит единичные символы или повторы (до 8 штук). Например, для всё тот же «baraban» LZSS сожмёт в «^{000110}bar2142n».

	T(n)	Пояснение
Best	$O(n)$	Длинные повторяющиеся последовательности
Average	$O(nb), b$ - размер буфера	Сканируем половину буфера
Worst	$O(nb)$	Повторов нет, каждый раз сканируем весь буфер
	S(n)	Пояснение
Best	$O(\log n)$	Длинные повторяющиеся последовательности
Average	$o(n)$	Среднее кол-во повторов
Worst	$O(n)$	Повторов нет

Таблица 8 – Кодирование LZSS

	T(n)	Пояснение
Best		Один проход по данным

Average	$O(n)$	
Worst		
	S(n)	Пояснение
Best	$O(n)$	Повторов нет
Average	$o(2^n)$	Среднее кол-во повторов
Worst	$O(2^n)$	Максимальное кол-во повторов

Таблица 9 – Декодирование LZSS

LZW

LZW – усовершенствованная версия LZ78, которые в отличие от LS77 и LZSS кодируют не «горизонтально», а «вертикально». Словарь LZW заранее заполнен всеми различными символами, на выходе только числа, новые фразы добавляются «на ходу».

Рассмотрим кодирование на примере слова «kukushka». Словарь уже проинициализирован «a» = 0, «h» = 1, «k» = 2, «s» = 3, «u» = 4. Читаем «k» - такая фраза есть, заносим в результат её индекс, запоминаем её. Следующая буква «u». В словаре нет фразы «u» - добавляем её, но не забываем записать код «u». Следующая фраза нам известна – «ku», так же записываем её индекс. Продолжаем, пока не дойдём до конца строки. В итоге получаем последовательность цифр «2353120» и словарь «a» = 0, «h» = 1, «k» = 2, «s» = 3, «u» = 4, «ku» = 5, «kus» - 6, «sh» - 7, «hk» - 8, «ka» - 9. То есть, к предыдущей известной фразе добавляем новую букву и получаем новую комбинацию, которая сохраняется в словарь.

	T(n)	Пояснение
Best	$O(n)$	Минимальное кол-во новых записей в словаре
Average	$O(n)$	При использовании словаря, каждый символ обрабатывается за $O(1)$

Worst	$O(n \log d), d$ – максимальный размер словаря	Если словарь построен на основе хеш-таблицы, возможны коллизии
	S(n)	Пояснение
Best	$O(n + d)$	Хранение результата и словаря
Average		
Worst		

Таблица 10 – Кодирование и декодирование LZW

Экспериментальная часть

Исследуем зависимость энтропии от размера блока, на которые разбивается текст, подаваемый на вход BWT и MTF, для тестовых файлов (рис. 3). Изначальная энтропия равнялась $\approx 7{,}3$, по мере увеличения размера блока значение экспоненциально убывает. Скорость обработки, если использовать реализацию с суффиксным массивом, примерно одинакова для всех случаев, поэтому размер буфера возьмём значение, после которого энтропия меняется незначительно – 16Кб.

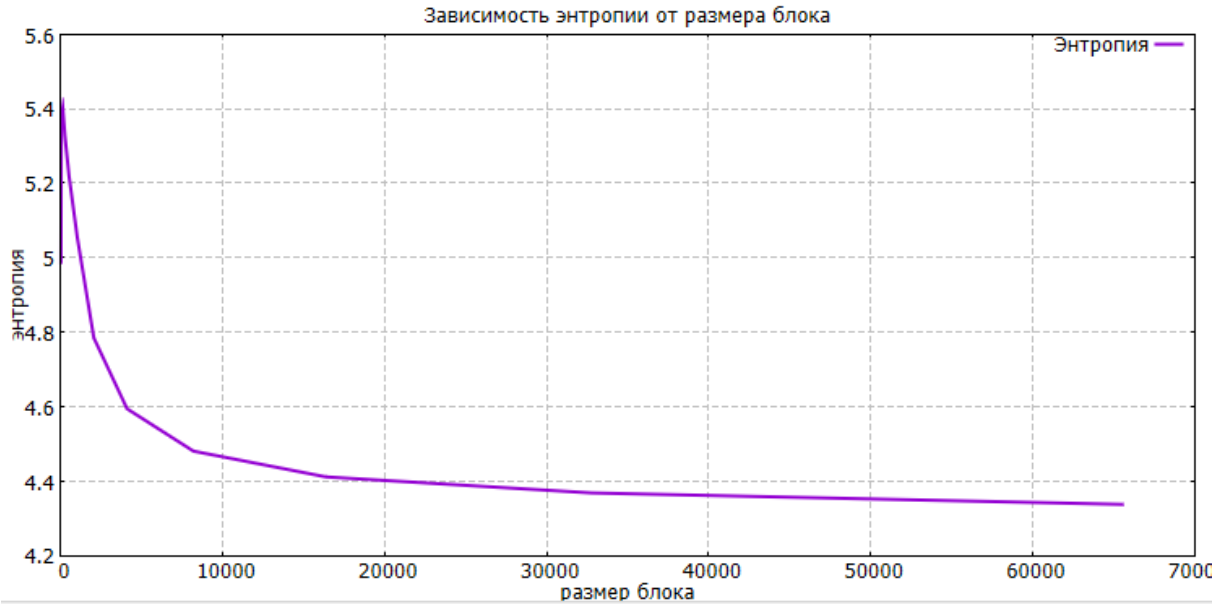


Рисунок 3

Теперь определим как размер буфера влияет на качество сжатия LZSS. При увеличении объёма буфера, коэффициент сжатия экспоненциально

возрастает (рис. 4), причём, с изменением объёма в 2 раза, скорость обработки также падает в 2 раза. Поэтому оптимальный размер – 16Кб.

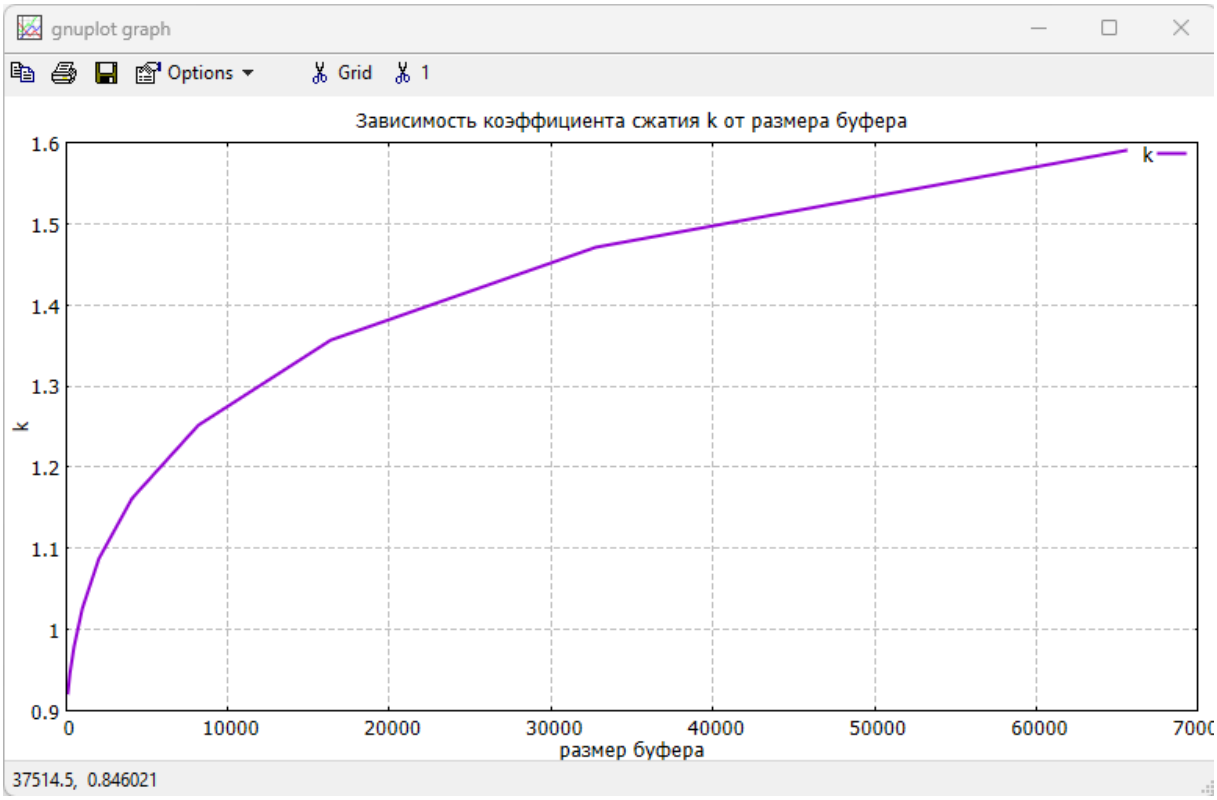


Рисунок 4

Точно так же определим зависимость коэффициента сжатия от максимально размера словаря для алгоритма LZW. Из графика (рис. 5) видно, что коэффициент так же увеличивается экспоненциально. Скорость обработки для любого размера словаря была одинаковой. В реализации LZW использовалось 2-х байтовое число для хранения номера фразы, следовательно, выбираем максимально возможную длину словаря – 64Кб.

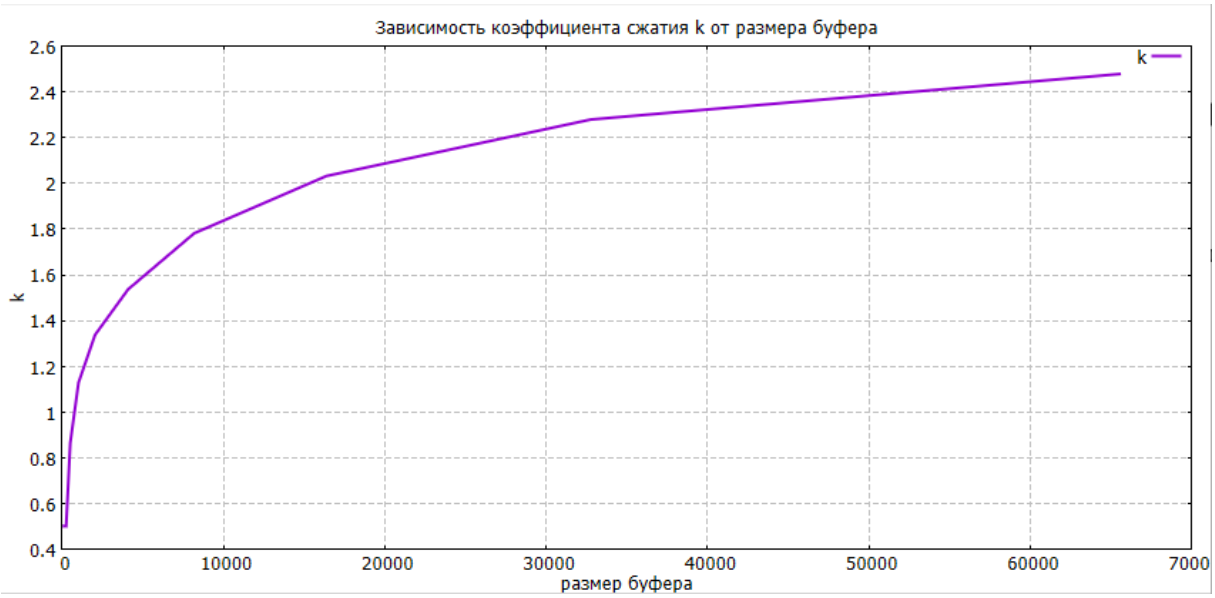


Рисунок 5

На основе всех рассмотренных алгоритмов сжатия скомпилируем 9 компрессоров:

1. Huffman
2. RLE
3. BWT + RLE
4. BWT + MTF + Huffman
5. BWT + MTF + RLE + Huffman
6. LZSS
7. LZSS + Huffman
8. LZW
9. LZW + Huffman

Проверим работу компрессоров на 6-и файлах: enwik-7 (текстовый файл с 10млн символов), бинарный файл размером 3Мб, raw изображения в чёрно-белом, цветном формате и оттенках серого 1280×960, и текст на русском языке 200Кб. Результаты сжатия занесены в таблицу (табл. 11).

Тип файла	Исходный размер, байты	Размер после компрессии, байты	Размер после декомпрессии, байты	Коэффициент сжатия
1. Huffman algorithm				

enwik7	10 000 000	6 412 790	10 000 000	1,56
.exe	2 823 952	2 602 256	2 823 952	1,09
raw black	153 600	118 949	153 600	1,29
raw gray	1 228 800	1 129 652	1 228 800	1,09
raw rgb	3 686 400	3 517 135	3 686 400	1,05
rus.txt	234 139	136 401	234 139	1,71
2. RLE				
enwik7	10 000 000	18 839 038	10 000 000	0,53
.exe	2 823 952	5 186 088	2 823 952	0,54
raw black	153 600	270 388	153 600	0,57
raw gray	1 228 800	1 613 642	1 228 800	0,76
raw rgb	3 686 400	6 429 266	3 686 400	0,57
rus.txt	234 139	437 902	234 139	0,53
3. BWT+RLE				
enwik7	10 000 000	10 074 060	10 000 000	0,99
.exe	2 823 952	3 715 060	2 823 952	0,76
raw black	153 600	275 328	153 600	0,56
raw gray	1 228 800	1 724 454	1 228 800	0,71
raw rgb	3 686 400	3 837 868	3 686 400	0,96
rus.txt	234 139	284 740	234 139	0,82
4. BWT + MTF + Huffman				
enwik7	10 000 000	4 137 993	10 000 000	2,42
.exe	2 823 952	1 682 027	2 823 952	1,68
raw black	153 600	115 307	153 600	1,33
raw gray	1 228 800	682 975	1 228 800	1,80
raw rgb	3 686 400	2 005 003	3 686 400	1,81
rus.txt	234 139	105 900	234 139	2,21
5. BWT + MTF + RLE + Huffman				
enwik7	10 000 000	4 931 758	10 000 000	2,03

.exe	2 823 952	1 927 248	2 823 952	1,47
raw black	153 600	142 706	153 600	1,08
raw gray	1 228 800	795 164	1 228 800	1,54
raw rgb	3 686 400	2 265 080	3 686 400	1,63
rus.txt	234 139	127 155	234 139	1,84
6. LZSS				
enwik7	10 000 000	6 459 837	10 000 000	1,55
.exe	2 823 952	2 436 848	2 823 952	1,16
raw black	153 600	161 522	153 600	0,95
raw gray	1 228 800	1 039 916	1 228 800	1,18
raw rgb	3 686 400	2 658 947	3 686 400	1,39
rus.txt	234 139	174 744	234 139	1,34
7. LZSS + Huffman				
enwik7	10 000 000	5 176 533	10 000 000	1,93
.exe	2 823 952	2 125 009	2 823 952	1,33
raw black	153 600	124 746	153 600	1,23
raw gray	1 228 800	917 229	1 228 800	1,34
raw rgb	3 686 400	2 334 971	3 686 400	1,58
rus.txt	234 139	137 698	234 139	1,70
8. LZW				
enwik7	10 000 000	4 665 542	10 000 000	2,14
.exe	2 823 952	4 159 574	2 823 952	0,68
raw black	153 600	135 808	153 600	1,13
raw gray	1 228 800	998 206	1 228 800	1,23
raw rgb	3 686 400	5 033 710	3 686 400	0,73
rus.txt	234 139	114 688	234 139	2,04
9. LZW + HA				
enwik7	10 000 000	4 636 813	10 000 000	2,16
.exe	2 823 952	3 089 308	2 823 952	0,91

raw black	153 600	128 913	153 600	1,19
raw gray	1 228 800	926 899	1 228 800	1,33
raw rgb	3 686 400	3 595 631	3 686 400	1,02
rus.txt	234 139	110 470	234 139	2,12

Таблица 11

Выводы

В ходе выполнения лабораторной работы были реализованы и исследованы основные алгоритмы сжатия данных: RLE, LZSS, LZW, MTF, BWT и алгоритм Хаффмана. Исследование показало, что алгоритм RLE эффективен только для данных с длинными сериями повторяющихся байтов. Алгоритм LZSS показал сильную зависимость от размера скользящего окна. Оптимальный размер буфера составил 16Кб. Для LZW оптимальный размер словаря составил 32-64Кб. Алгоритм BWT показал, что оптимальный размер блока составляет 16КБ. По результатам тестирования на шести различных файлах наилучшие результаты показала комбинация BWT+MTF+Хаффмана, обеспечившая коэффициент сжатия от 2,2 для текстовых файлов до 1,6 для случайных бинарных данных. LZSS+Хаффман показал стабильно хорошие результаты для всех типов данных с коэффициентом сжатия 1,3 до 2. Все реализованные алгоритмы успешно справились с обратным преобразованием.